Plan maestro de pruebas de integración

Proyecto: NextGuard · Monitoreo IoT: ESP32 (DHT11 + MQ135) → API Python → InfluxDB + PostgreSQL → Grafana
Entorno de ejecución: AWS Learner Lab, una EC2 con un contenedor Docker por servicio (api, influxdb, postgres, grafana)
Versión: 2.0 · **Fecha:** 2026-10-17 · **Idioma:** español

---

1. Propósito y alcance

Este plan define cómo se comprobará que los componentes del sistema funcionan **juntos**: que una lectura tomada por los sensores conectados a un ESP32 llega por WiFi a una API en Python, se valida y queda guardada, sin pérdidas ni duplicados, en una base de series de tiempo (InfluxDB) y que Grafana la muestra en un dashboard por cliente y sensor. La configuración del sistema (clientes, sensores, alertas) vive en PostgreSQL.

La arquitectura sigue la propuesta de la clase «Del sensor a Grafana» (SIY6122): **sensor con portal cautivo → API (único punto de entrada) → InfluxDB 2.7 + PostgreSQL → Grafana**, todo en una EC2 con `docker compose` y solo dos puertos públicos (API :8000 y Grafana :3000).

Dentro del alcance

- Sensores DHT11 (temperatura y humedad, módulo HW-036) y MQ135 (calidad del aire, salida analógica con divisor de tensión) conectados al ESP32.
- Firmware del ESP32: portal cautivo (WiFiManager) con configuración en flash, muestreo, cola en RAM, reintentos, hora NTP y envío HTTP(S).
- API de ingesta (FastAPI): contrato de la clase `{sensor, cliente, ubicacion, tipo, valor, unidad}` más extensión (varias mediciones, `seq`, lotes), validaciones y gestión de clientes y sensores.
- InfluxDB 2.7 (lecturas) y PostgreSQL 16 (clientes, sensores, alertas y marcas de idempotencia); SQLite y almacenamiento en memoria solo para las pruebas locales.
- Dashboards de Grafana en Flux con variables cliente y sensor (datasource y dashboard aprovisionados, token de solo lectura).
- Despliegue en AWS Learner Lab: una EC2 con Docker Compose, Security Group, Elastic IP.
- Pruebas no funcionales básicas (latencia, carga ligera, recuperación, ~3 h de operación continua) y seguridad básica.

Fuera del alcance (primera entrega)

- Medición certificada de CO_2/gases con el MQ135: se reporta `co2_equiv` (ppm), una estimación con la curva típica y R0 autocalibrado en aire limpio, útil para tendencias; `calidad_aire_raw` queda como diagnóstico.
- Alertas por correo/Telegram y portal de clientes (semana 15 del cronograma); el modelo `alertas` ya existe en PostgreSQL.
- Alta disponibilidad (varias instancias, balanceadores) y pruebas de penetración formales.
- Despliegue automatizado con Ansible (semana 14).

---

2. Sistema bajo prueba

2.1 Arquitectura

```
Sensores DHT11 + MQ135 ─► ESP32 ── WiFi ──► Internet ──► EC2 (docker compose)
                          portal cautivo                  ├─ api · FastAPI :8000 ──► influxdb
2.7   (lecturas)
                          cola en RAM · NTP               │   valida · API key  └──► postgres
16    (clientes, sensores,
                          reintentos                      │   idempotencia por seq
alertas, marcas de seq)
                                                          └─ grafana :3000 ──(token solo
lectura)──► influxdb
```

Las bases no publican puertos: solo se accede a ellas desde la red interna de Docker. Diagramas detallados en `diagramas/index.html` (arquitectura, flujo de datos, secuencia de pruebas, despliegue AWS, niveles y Gantt).

2.2 Contrato de la API

Método y ruta	Uso	Respuestas
`POST /api/v1/lecturas`	Una lectura: formato de la clase `{sensor, cliente, ubicacion, tipo, valor, unidad}` o varias mediciones `{…, mediciones:[{tipo, valor, unidad}], seq, timestamp, rssi, fw_version}`	`201` guardada · `200` duplicada (idempotente) · `401` clave · `403` cliente o sensor no registrado · `413` demasiado grande · `422` formato, rango físico o fecha futura · `503` almacén no disponible
`POST /api/v1/lecturas/batch`	Hasta 100 lecturas (cola acumulada)	`200` con `{insertadas, duplicadas}` · mismos errores
`GET /api/v1/lecturas`	Consulta por `cliente`, `sensor`, `desde`, `hasta`, `limit` (24 h por defecto)	`200` · `401` · `503`
`POST /clientes` · `GET /clientes` · `GET /clientes/{id}`	Gestión de clientes	`201` · `409` repetido · `404`

Método y ruta	Uso	Respuestas
`GET /clientes/{id}/sensores` · `GET /clientes/{id}/sensores/{sensor}/lecturas`	Sensores del cliente (con última lectura) y sus lecturas	`200` · `404`
`GET /health`	Estado de la API, PostgreSQL e InfluxDB	`200` · `503` con la base caída

Autenticación: cabecera `X-API-Key` en todos los endpoints salvo `/health`. Identificador de idempotencia: par (`sensor`, `seq`) por cliente. Validación de rango físico: temperatura −40…80 °C, humedad 0…100 %, `co2_equiv` 300…10000 ppm, `calidad_aire_mv` 0…3300 mV (diagnóstico), `calidad_aire_raw` 0…4095; `timestamp` no puede superar 5 min en el futuro. Toda la API trabaja en UTC.

2.3 Modelo de datos

InfluxDB 2.7 (org `duoc`, bucket `sensores`): measurement = tipo de medición (`temperatura`, `humedad`, `co2_equiv`…); tags `sensor`, `cliente`, `ubicacion`, `unidad`; fields `valor`, `seq`, `rssi`, `ts_dispositivo` y `fw`; hora del punto = hora del servidor. Agregar una magnitud nueva **no requiere migrar nada** (casos `IT-DB-04`, `IT-E2E-07`). Tokens: la API recibe uno de lectura y escritura y Grafana uno de **solo lectura**, creados al inicializar el volumen (`db/influx-init.sh`).

PostgreSQL 16 (`db/init.sql`):

- `clientes` — `id_cliente` (clave), nombre y correo.
- `sensores` — un registro por (cliente, sensor), con ubicación, tipo, ciclo y última lectura; se auto-enrola con su primera lectura.
- `alertas` — reglas por sensor (magnitud, condición, umbral, duración, destinatario, canal); se usarán en la semana 15.
- `lecturas_vistas` — clave primaria (`sensor_id`, `seq`): garantiza la idempotencia aunque dos reintentos lleguen a la vez.

2.4 Decisiones de diseño que las pruebas deben confirmar

Decisión	Razón	Casos que la verifican
InfluxDB para lecturas y PostgreSQL para configuración	Sigue la arquitectura de la clase; la TSDB escribe y consulta por rango y la relacional guarda quién es quién	IT-DB-01, IT-DB-15…17, IT-API-30
InfluxDB 2.x (no 3 Core)	Permite un token por bucket de **solo lectura** para Grafana y no limita las consultas a ~72 h	IT-DB-18, IT-ENV-04
`seq` por sensor, persistido en NVS con salto de 20 al reiniciar; marca única en PostgreSQL	Un reintento no duplica y un reinicio no reutiliza números	IT-API-02, IT-DB-02, IT-E2E-03
Si InfluxDB falla se libera la marca de seq y se responde 503	El reintento del ESP32 no se toma por duplicado	IT-DB-12, IT-DB-13

Decisión	Razón	Casos que la verifican
Cola en RAM de 60 muestras, envío por lotes	Sobrevivir a cortes de red de hasta ~10 min	IT-API-20, IT-NET-03, IT-NFR-08
Portal cautivo con configuración en flash	Un dispositivo se reconfigura sin recompilar	IT-NET-06…08
MQ135 en ADC1 con divisor de tensión y precalentamiento	ADC2 no funciona con WiFi y el módulo entrega hasta 5 V	IT-SEN-07, IT-SEN-08, IT-API-29
Bases sin puertos publicados; token de Grafana de solo lectura	Superficie de ataque mínima	IT-ENV-03, IT-ENV-04, IT-SEC-06, IT-DB-18
Un contenedor por servicio con `restart: unless-stopped` y volúmenes nombrados	El stack vuelve solo tras reiniciar la EC2 y no pierde datos	IT-ENV-05, IT-ENV-08, IT-NFR-07
Validación estricta (422/403), rango físico y API key	La API no confía en el dispositivo	IT-API-03…10, IT-API-22…29, IT-SEC-01…05, IT-SEC-10/11

---

3. Requisitos a verificar

ID	Requisito
RF-01	El sistema mide temperatura y humedad con el sensor conectado al ESP32.
RF-02	El ESP32 se conecta a la red WiFi y recupera la conexión tras una caída.
RF-03	El ESP32 envía cada lectura a la API en JSON, autenticado con API key.
RF-04	La API valida el contenido y persiste las lecturas en InfluxDB (series de tiempo) y la configuración en PostgreSQL.
RF-05	No se pierden ni se duplican lecturas ante reintentos, cortes o reinicios.
RF-06	Las lecturas pueden consultarse por cliente, sensor y rango de tiempo.
RF-07	Se pueden agregar nuevas magnitudes sin cambiar el esquema.
RF-08	Un dashboard de Grafana muestra las lecturas almacenadas (temperatura, humedad, calidad del aire, estado de los sensores).
RF-09	El dispositivo se configura sin recompilar mediante un portal cautivo (WiFi, servidor, cliente, sensor, ubicación, ciclo).
RF-10	Clientes y sensores se registran en PostgreSQL; la API rechaza lecturas de clientes no registrados y auto-enrola sensores.
RF-11	El sistema mide la calidad del aire (MQ135) además de temperatura y humedad (DHT11).
RNF-01	Latencia de entrega (ESP32 → BD) con p95 menor a 2 s en condiciones normales.
RNF-02	El sistema se recupera solo tras caídas de red, API, bases de datos o energía.
RNF-03	Seguridad básica: autenticación, validación de entradas, bases no expuestas, token de Grafana de solo lectura, sin secretos en el código.
RNF-04	Trazabilidad temporal: hora del dispositivo (NTP) y hora del servidor en cada lectura.

La matriz completa requisito ↔ caso está en `02_casos_de_prueba.md` y en la hoja *Trazabilidad* de `casos_de_prueba.xlsx`.

---

4. Estrategia de pruebas

4.1 Niveles

Nivel	Qué integra	Método	Casos
N0 Entorno	EC2, Docker Compose, Security Group, tokens de InfluxDB, Grafana	Manual + `smoke_env.sh`	IT-ENV-01…08
N1 Sensor ↔ ESP32	DHT11, MQ135 y firmware	Manual (monitor serie)	IT-SEN-01…10
N2 ESP32 ↔ red y API	WiFi, NTP, portal cautivo, contrato HTTP, validaciones, reintentos	pytest + simulador + manual	IT-NET-01…08, IT-API-01…32
N3 API ↔ bases	Persistencia en InfluxDB y PostgreSQL, idempotencia, caída de una base, token de solo lectura	pytest (memoria/SQLite y servicios reales) + manual	IT-DB-01…18
N4 Extremo a extremo	Sensores reales → InfluxDB → Grafana	Manual + simulador	IT-E2E-01…08
N5 No funcionales	Latencia, carga, recuperación, operación continua	Simulador + observación	IT-NFR-01…09
N6 Seguridad básica	Autenticación, inyección SQL/Flux, exposición, tokens	pytest + comprobación externa	IT-SEC-01…11

4.2 Enfoque: de abajo hacia arriba y sin depender del hardware

1. **Primero la API y las bases con datos simulados.** `simulador_sensor.py` reproduce el comportamiento del ESP32 (varios sensores, reenvíos duplicados, payloads inválidos, claves erróneas, MQ135 con `--with-mq`) y `pytest` fija el contrato. Así los defectos de servidor se descubren antes de conectar hardware.
2. **Después el ESP32 real** con los sensores simulados (`USE_DHT=0`, `USE_MQ135=2`) para aislar la red de los sensores; la configuración se hace desde el portal cautivo.
3. **Luego los sensores reales** (`USE_DHT=1`, `USE_MQ135=1`, con el divisor de tensión comprobado antes de conectar) y la prueba extremo a extremo hasta Grafana.
4. **Al final los fallos inducidos** (apagar router, detener la API, detener influxdb o postgres, cortar energía) y la operación continua.

4.3 Criterios de entrada y salida

Fase	Entrada (puede empezar si…)	Salida (termina cuando…)
N0	Laboratorio iniciado; EC2 permitida (t3.medium o t3.large)	`smoke_env.sh` sin fallas; IT-ENV-01…08 aprobados
N2/N3	N0 aprobado; API desplegada	Suite pytest 100 % en memoria/SQLite **y** contra PostgreSQL e InfluxDB reales; simulador con verificación OK

Fase	Entrada (puede empezar si…)	Salida (termina cuando…)
N1	Firmware compila; sensores cableados (MQ135 con divisor verificado)	IT-SEN-01…10 aprobados
N4	N1–N3 aprobados	IT-E2E-01…03 y 08 aprobados (0 faltantes, 0 duplicados)
N5/N6	N4 aprobado; ≥ 3,5 h de sesión para el soak	Ver criterios de aceptación (§8)

---

5. Entorno de pruebas

5.1 Recursos AWS (Learner Lab / sandbox)

Recurso	Configuración propuesta	Observaciones
EC2	Amazon Linux 2023, `t3.medium` (4 GB) como base; `t3.large` (8 GB) si se añaden servicios o para la carga IT-NFR-04. SG `nextguard-sg` (22 y 3000 solo «Mi IP», 8000 abierto al ESP32)	Ejecuta `docker compose` con los contenedores `api`, `influxdb`, `postgres` y `grafana`; **Elastic IP** si está permitida. Límites de memoria en compose: influxdb 768 MB, postgres 512 MB, api 512 MB, grafana 512 MB
InfluxDB	Contenedor `influxdb:2.7`, volumen `influx-data`, sin puertos publicados	Org `duoc`, bucket `sensores`; token de la API (lectura+escritura) y token de Grafana (solo lectura) en volúmenes separados
PostgreSQL	Contenedor `postgres:16-alpine`, volumen `pg-data`, sin puertos publicados	Base `iot`, usuario `iot`; tablas de `db/init.sql`
Security Group	`nextguard-sg`	Sin reglas para los puertos 8086 (InfluxDB) ni 5432 (PostgreSQL)
IAM	`LabRole` / `LabInstanceProfile`	No se pueden crear roles propios

La guía paso a paso está en `docs/deploy_ec2.md`.

5.2 Restricciones del laboratorio a tener presentes

> Verificar en la cuenta real: los servicios habilitados y las cuotas pueden variar entre laboratorios y sandboxes.

- **Sesión de ~4 h:** al vencer, las EC2 se detienen. La prueba de operación continua (3 h) debe iniciarse con al menos 3,5 h restantes.
- **IP pública variable** si no hay Elastic IP: cambiar `API_HOST` en `config.h` y volver a cargar el firmware en cada sesión.
- **Presupuesto limitado:** detener la EC2 al cerrar cada sesión.
- **Región habitual:** `us-east-1` o `us-west-2`.

5.3 Entorno local (PC)

Python 3.11+ con `requirements-dev.txt`; almacenamiento en memoria y SQLite. Con Docker también se pueden correr las pruebas contra InfluxDB y PostgreSQL reales (IT-DB-10). Permite ejecutar toda la suite y el simulador sin AWS.

5.4 Hardware

ESP32 (DevKit o similar) con cable USB de datos de calidad, sensor DHT11 (módulo HW-036, 3 pines S/+/−; con DHT22 solo cambia `DHT_TYPE`), sensor MQ135 con dos resistencias para el divisor de tensión (p. ej. 10 kΩ y 20 kΩ: la salida analógica puede superar los 3,3 V que tolera el GPIO34) y alimentación de 5 V para su calentador, protoboard y cables, router 2.4 GHz con WPA2, termómetro/higrómetro de referencia, segundo dispositivo para probar el hotspot (opcional).

5.5 Conexión WiFi (lección de la etapa de diagnóstico)

Durante la puesta en marcha el ESP32 no conectaba y la causa era una contraseña mal escrita. El diagnóstico con `wifi_diagnostico.ino` (escaneo + motivo de desconexión) permitió descartar banda, canal y señal. El escaneo también mostró **dos puntos de acceso con el mismo SSID** (posible repetidor); el firmware ya elige el de mejor señal. Este mismo mecanismo se reutiliza en los casos IT-NET-01 y IT-NET-02.

---

6. Herramientas y scripts

Elemento	Ubicación	Función
API FastAPI	`api/` (con `Dockerfile`)	Sistema bajo prueba
Stack Docker	`docker-compose.yml`	Servicios `api`, `influxdb`, `postgres` y `grafana` en una EC2
Dashboards	`grafana/`	Datasource InfluxDB (Flux, token de solo lectura) y dashboard «NextGuard · Monitoreo IoT» con variables cliente y sensor
Esquemas	`db/init.sql`, `db/influx-init.sh`	DDL de PostgreSQL y creación de los tokens de InfluxDB
Suite pytest	`tests/`	Contrato, validaciones, idempotencia, seguridad, carga ligera e integración con InfluxDB (`pytest`)
Simulador	`scripts/simulador_sensor.py`	Genera tráfico; `--verify` compara lo enviado con lo guardado
Prueba de humo	`scripts/smoke_env.sh`	Verifica API, validaciones, Grafana y que InfluxDB y PostgreSQL no sean accesibles desde fuera
Colección Postman	`docs/postman/`	13 peticiones con 19 aserciones (semana 12-13); `npx newman run … --delay-request 1200`

Elemento	Ubicación	Función
Firmware	`firmware/esp32_envio/`	ESP32 con portal cautivo, DHT11, MQ135, cola, NTP y reintentos
Diagnóstico WiFi	`firmware/wifi_diagnostico/`	Escaneo y motivo de desconexión
Generador de documentos	`scripts/generar_documentos.py`	Crea .md, .csv y .xlsx desde `casos_de_prueba.py`

Comandos frecuentes:

```
# Suite completa en local (memoria + SQLite): 49 pruebas, 3 de integración se omiten
pytest
# Stack completo en la EC2 (o en tu PC con Docker)
cp .env.example .env && docker compose up -d --build
# Suite contra PostgreSQL e InfluxDB reales (52 pruebas): desde un contenedor en la red de
compose
#   TEST_DATABASE_URL=postgresql+psycopg://iot:<clave>@postgres:5432/iot_test
TEST_INFLUX_URL=http://influxdb:8086  TEST_INFLUX_TOKEN=<token de la API>
# Tráfico simulado con verificación
python scripts/simulador_sensor.py --url http://<IP>:8000 --api-key <clave> --devices 3 --count
50 --dup-rate 0.1 --with-mq --verify
```

---

7. Plan de ejecución

Cada sesión de laboratorio dura ~4 h; se planifican **sesiones cortas y acotadas** para no depender de recursos que expiran.

Sesión	Objetivo	Casos	Duración estimada
S0 *(PC, sin AWS)*	Suite pytest y simulador con almacenamiento en memoria; stack local con Docker	IT-API-01…17, 22…32, IT-DB-01…09, 12…14, IT-SEC-01…05, 10/11, IT-NFR-01/02	30 min
S1	Crear SG y EC2; instalar Docker; `docker compose up`; `smoke_env.sh`; revisar tokens y Grafana	IT-ENV-01…08, IT-DB-18, IT-SEC-06	2–3 h
S2	Suite pytest contra PostgreSQL e InfluxDB; simulador desde fuera de AWS; colección Postman	IT-DB-10, 15…17, IT-NFR-03, IT-E2E-04/06/07	1–2 h
S3	Portal cautivo y firmware con sensores simulados; luego DHT11 y MQ135 reales (divisor verificado)	IT-NET-06…08, IT-SEN-01…10, IT-NET-01/02/04, IT-API-18/19	2–3 h
S4	Prueba extremo a extremo con ESP32 real hasta el dashboard	IT-E2E-01…03/05/08	1 h
S5	Fallos inducidos y recuperación	IT-NET-03/08, IT-API-20/21, IT-DB-11, IT-NFR-06/07/08	2 h
S6	Carga y seguridad	IT-NFR-04, IT-NET-05, IT-SEC-07/08/09	1–2 h

Sesión	Objetivo	Casos	Duración estimada
S7	Operación continua de 3 h y cierre	IT-NFR-05, IT-NFR-09	3,5 h

Al terminar cada sesión: guardar evidencias, actualizar `casos_de_prueba.xlsx`, detener la EC2.

---

8. Criterios de aceptación

El sistema se considera aceptado para la primera entrega si se cumple **todo** lo siguiente:

#	Criterio	Métrica
1	Casos de prioridad Alta	100 % aprobados
2	Casos de prioridad Media	≥ 90 % aprobados; los restantes con defecto registrado y plan
3	Integridad de datos	0 lecturas duplicadas y 0 faltantes en IT-E2E-01 y en la operación continua (tolerancia: ≤ 1 % de pérdida solo en la operación continua)
4	Latencia	p95 < 2 s medida desde fuera de AWS (IT-NFR-03)
5	Recuperación	El sistema retoma el envío por sí solo tras caída de router, API, InfluxDB y PostgreSQL (IT-NET-03, IT-API-20, IT-DB-11, IT-DB-12)
6	Seguridad	InfluxDB y PostgreSQL no accesibles desde Internet; token de Grafana sin permiso de escritura; sin secretos en el código; 0 inyecciones SQL/Flux exitosas
7	Defectos	0 abiertos de severidad 1 o 2
8	Evidencia	Cada caso ejecutado con resultado real y evidencia en la matriz

8.1 Severidad de defectos

Severidad	Definición	Ejemplo
1 – Crítica	Pérdida o corrupción de datos, exposición de secretos	Lecturas duplicadas; puerto 8086 o 5432 publicado
2 – Alta	Función principal inutilizable	La API responde 500 a una lectura válida
3 – Media	Función degradada con alternativa	Reintento sin backoff
4 – Baja	Cosmético o de documentación	Mensaje poco claro

Cada defecto se registra con: ID de caso, pasos, resultado esperado vs real, evidencia (captura del monitor serie, log de `docker compose logs`, consulta SQL o Flux), severidad y estado.

---

9. Riesgos

ID	Riesgo	Prob.	Impacto	Mitigación
R-01	La IP pública de la EC2 cambia al reiniciar y el ESP32 deja de enviar	Alta	Alto	Elastic IP; si no es posible, actualizar `config.h` y documentar el procedimiento
R-02	La sesión del laboratorio vence durante la operación continua	Media	Alto	Iniciar con ≥ 3,5 h restantes; registrar la hora de inicio; reanudar y comparar por `seq`
R-03	Presupuesto del laboratorio agotado	Media	Alto	Detener la EC2 al cerrar; usar `t3.medium` y subir a `t3.large` solo si hace falta
R-04	El tipo de instancia (`t3.medium` / `t3.large`) o Docker no están disponibles en el sandbox	Media	Alto	Verificar en S1; alternativa: `t3.small` con límites de memoria menores y sin Grafana durante la carga
R-13	La EC2 queda sin memoria al correr api, influxdb, postgres y grafana juntos (carga IT-NFR-04)	Media	Medio	`mem_limit` por contenedor; observar con `docker stats`; subir a `t3.large`
R-14	`docker compose down -v` borra los datos	Baja	Alto	Documentado en `docs/deploy_ec2.md`: usar `down` sin `-v`; exportar resultados antes de cerrar
R-15	Doble escritura no atómica: la marca de seq (PostgreSQL) y el punto (InfluxDB) pueden quedar desfasados si la API cae entre ambas	Baja	Medio	Si InfluxDB falla se borra la marca y se responde 503 (IT-DB-12); queda una ventana mínima si el proceso muere justo entre ambos pasos; el simulador `--verify` y el conteo por seq la detectan
R-16	InfluxDB 2.x tarda unas décimas de segundo en hacer visible una escritura	Alta	Bajo	La idempotencia no depende de InfluxDB (usa PostgreSQL); las verificaciones consultan con reintentos hasta 10 s
R-17	El token de Grafana o de la API se filtra	Baja	Alto	Tokens con privilegios mínimos, en volúmenes separados y fuera del repositorio; el de Grafana no puede escribir (IT-DB-18); rotar con `docker compose down -v` solo si se acepta perder datos
R-18	MQ135 con lecturas relativas, sin calibrar, y salida analógica superior a 3,3 V	Media	Alto	Reportar `co2_equiv` (ppm estimado, R0 autocalibrado) y `calidad_aire_raw`; divisor de tensión verificado con multímetro antes de conectar (IT-SEN-07); precalentamiento (IT-SEN-08)
R-19	Los materiales de la clase muestran ClickHouse en el demo y InfluxDB + PostgreSQL en las diapositivas	Media	Medio	Se siguió la diapositiva 4 (InfluxDB + PostgreSQL); confirmar con el docente antes de la Evaluación 2
R-05	Lecturas `NaN` o poco precisas del DHT (especialmente DHT11)	Media	Medio	El firmware descarta `NaN`; comparar con instrumento de referencia; registrar la tolerancia del sensor
R-06	Caídas de tensión (cable/puerto USB) provocan fallos de WiFi	Media	Medio	Cable de datos corto y de calidad; puerto directo; alimentación externa; `USE_LOW_TX_POWER` en el diagnóstico
R-07	Cobertura 2.4 GHz insuficiente o repetidor con el mismo SSID	Media	Medio	Firmware con selección por mejor señal; medir con IT-NET-05

ID	Riesgo	Prob.	Impacto	Mitigación
R-08	Tráfico HTTP sin cifrar (API key visible en la red)	Alta	Medio	Riesgo aceptado en el laboratorio; fase 2 con HTTPS; documentar en IT-SEC-09
R-09	Corte de energía pierde la cola en RAM (hasta 60 muestras)	Media	Bajo	Documentar la pérdida máxima; opción futura: persistir la cola en flash
R-10	Reloj sin NTP genera lecturas sin `ts_dispositivo`	Media	Bajo	La API acepta `null` y usa `ts_servidor`; IT-NET-04
R-11	Secretos expuestos (contraseña WiFi, API key, credenciales de BD, claves de Grafana)	Media	Alto	`config.h` y `.env` fuera del repositorio (el repo es público); el portal cautivo guarda WiFi y API key en la flash del ESP32; rotar la clave WiFi que se compartió durante el diagnóstico; IT-SEC-07
R-12	El salto de `seq` tras un reinicio se interpreta como pérdida	Baja	Bajo	Documentado: el salto es de hasta 20 por diseño; los análisis de integridad distinguen reinicios

---

10. Estado de verificación al momento de emitir este plan

Para no confundir lo **diseñado** con lo **comprobado**:

Elemento	Estado
Suite pytest con almacenamiento en memoria y SQLite (49 pruebas, 3 de integración omitidas)	✅ Ejecutada; todas pasan
Suite pytest contra PostgreSQL 16 e InfluxDB 2.7 reales (52 pruebas)	✅ Ejecutada desde un contenedor en la red de compose; todas pasan
Stack `docker compose` (api + InfluxDB + PostgreSQL + Grafana) en local	✅ Levantado con Docker; `/health` ok con ambas bases; tokens creados por `db/influx-init.sh`
Token de Grafana de solo lectura	✅ Escribir con él devuelve 403; con el token de la API, 204
Dashboard de Grafana en Flux	✅ Los 9 paneles y las 2 variables responden 200 vía `/api/ds/query`
Simulador con `--verify` (reintentos 20 %, casos inválidos 20 %, MQ135)	✅ 3 sensores, 0 faltantes y 0 duplicados
`smoke_env.sh` contra el stack local	✅ 9 de 9 comprobaciones; `--externo` ✅
Colección Postman (newman)	✅ 13 peticiones, 19 aserciones, 0 fallos
Firmware `esp32_envio.ino` (portal cautivo, DHT11, MQ135)	✅ Compila para `esp32:esp32:esp32` en sus 4 combinaciones de sensores (81 % de la flash); ⏳ sin probar en el ESP32 con el portal
Despliegue de esta versión en la EC2 (t3.medium, Elastic IP)	✅ 2026-10-17: stack de 4 contenedores healthy; `smoke_env.sh` 9/9 desde fuera de AWS; `--externo`: 8086 y 5432 cerrados; simulador `--verify` (60 lecturas, 0 faltantes, 0 duplicados, p95 261 ms); token de Grafana → 403 al escribir; Postman 19/19 contra la EC2

Elemento	Estado
Reinicio de la EC2 con la nueva versión (IT-ENV-05, IT-ENV-08, IT-NFR-07)	✅ Los 4 contenedores vuelven solos (≈10 s tras el arranque); 60 lecturas antes y 60 después; datasource y dashboard de Grafana intactos
Pruebas con hardware (portal, DHT11, MQ135, E2E hasta Grafana) y sesiones S1–S7	⏳ Pendiente
Matriz `.xlsx`	✅ Regenerada desde `scripts/casos_de_prueba.py`: 104 casos

---

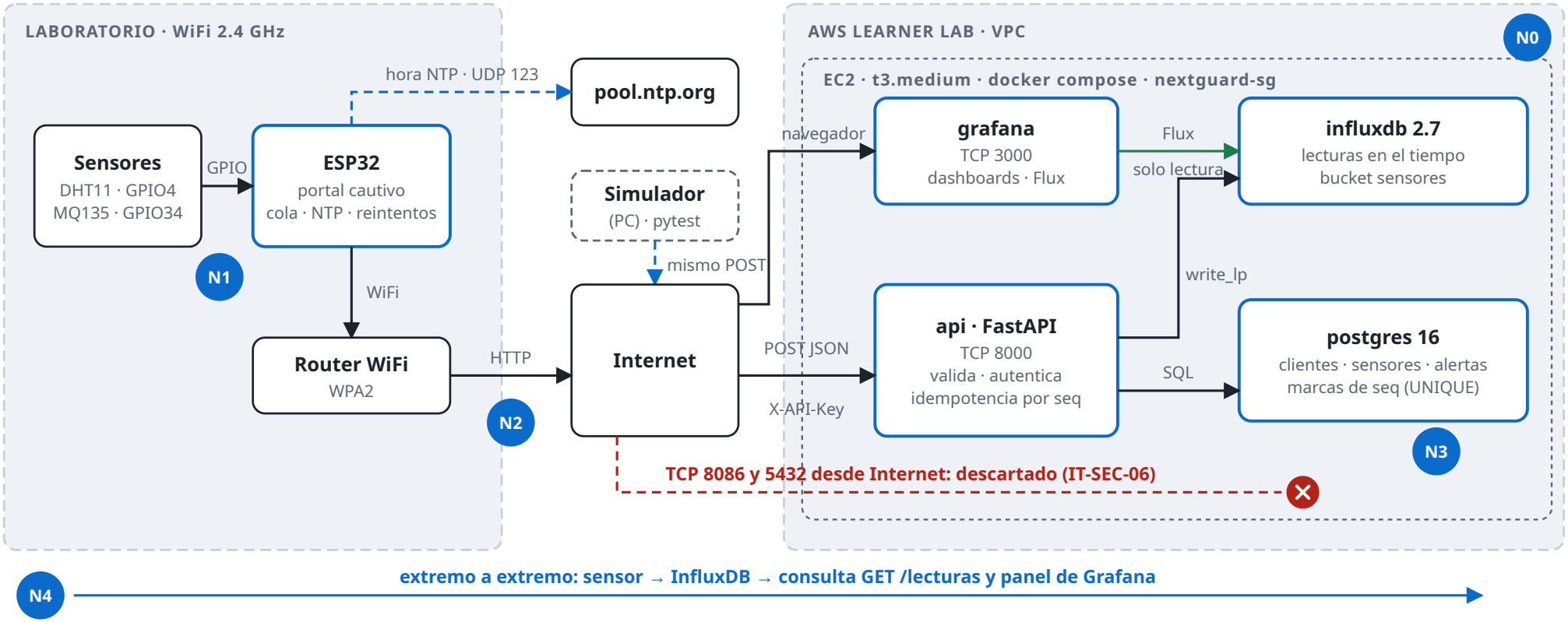
11. Entregables del paquete

```
NextGuard-monitoreo-IoT/
├── README.md                    índice y cómo empezar
├── api/                         API FastAPI + Dockerfile
├── docker-compose.yml           api, influxdb, postgres y grafana
├── .env.example                 variables del stack (copiar a .env)
├── db/                          init.sql (PostgreSQL) e influx-init.sh (tokens de InfluxDB)
├── grafana/                     datasource y dashboards aprovisionados (Flux)
├── firmware/                    esp32_envio y wifi_diagnostico (config.h.example)
├── tests/                       pytest
├── scripts/                     simulador, humo y generadores de documentos
└── docs/                        este plan (.md/.pdf), casos de prueba, deploy_ec2.md, postman/
y diagramas/ (HTML + PDF, incluye el Gantt)
```

12. Aprobaciones

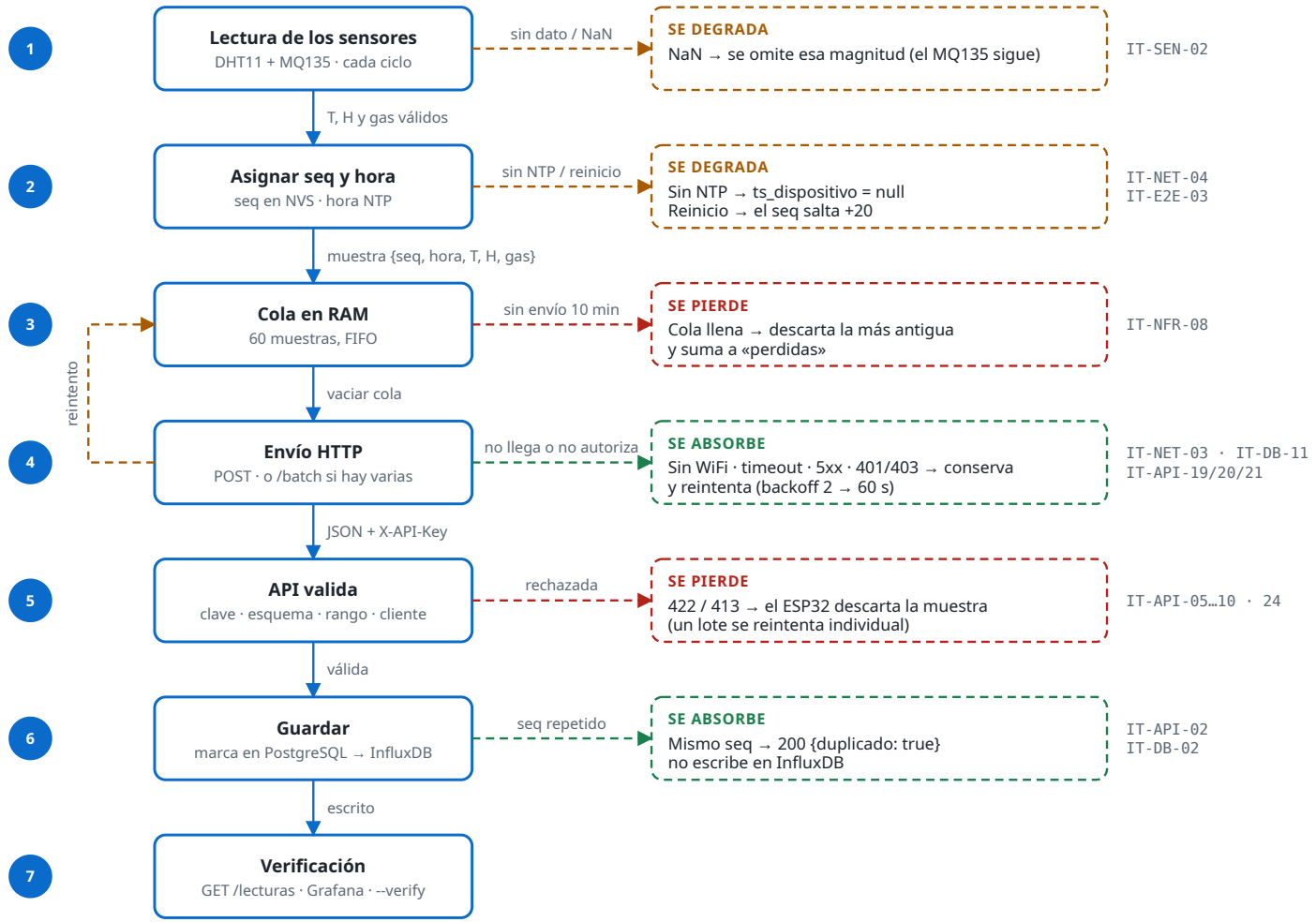
Rol	Nombre	Fecha	Firma
Responsable del proyecto			
Docente / revisor			

Anexo A.1 · Arquitectura bajo prueba



El ESP32 solo conoce a la API. La API es el único componente que escribe: las lecturas en InfluxDB y los clientes, sensores y marcas de seq en PostgreSQL. Grafana consulta InfluxDB con un token de **solo lectura**. La línea roja muestra el camino que **debe fallar**: las bases no publican sus puertos.

Anexo A.2 · Recorrido de una lectura



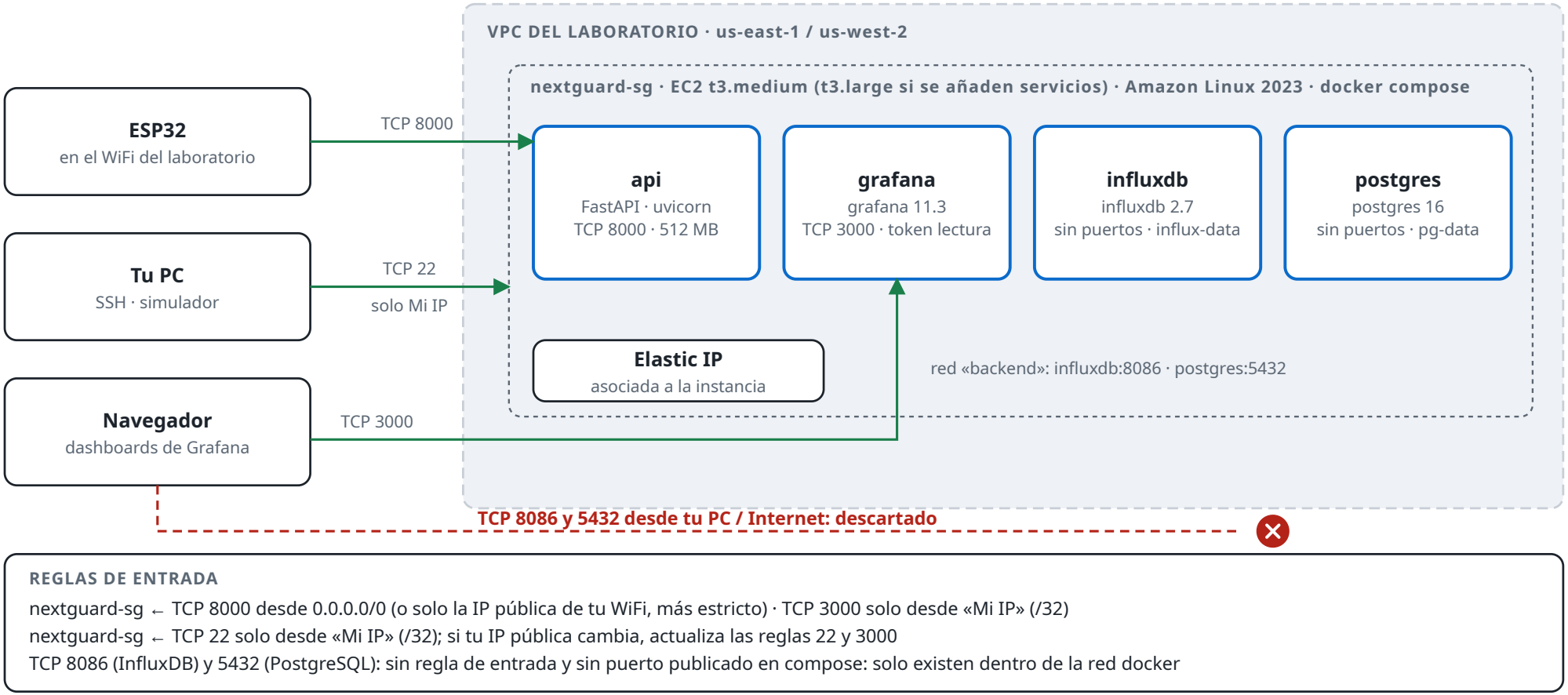
Cada recuadro punteado es un fallo inducido en las pruebas. **Se absorbe**: no se pierde nada. **Se degrada**: la lectura llega con menos información. **Se pierde**: una muestra se descarta y queda contada.

Anexo A.3 · Secuencia de las pruebas



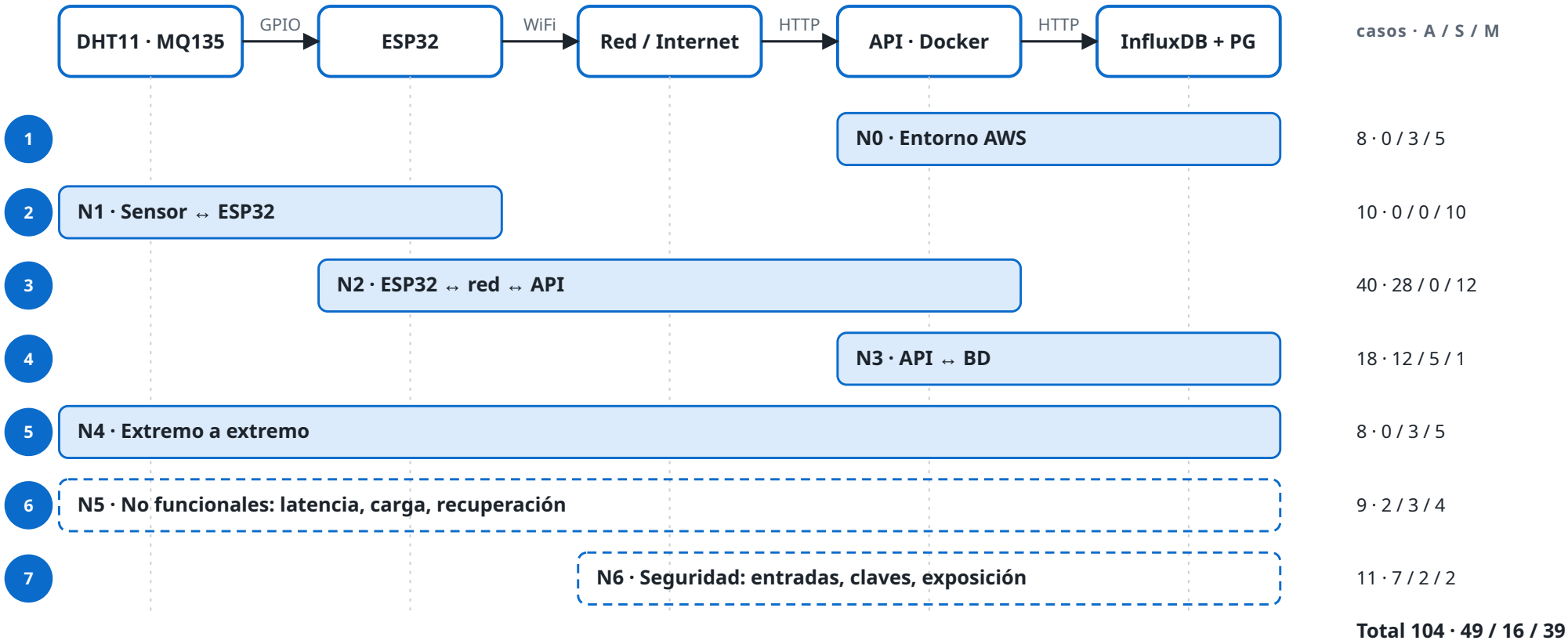
Las líneas discontinuas son respuestas. Verde = éxito, ámbar = degradación recuperable, rojo = rechazo. El escenario D es el que justifica la cola en el ESP32, el endpoint `/batch` y la liberación de la marca de seq.

Anexo A.4 · Despliegue en AWS



Flechas verdes = conexiones permitidas. La línea roja discontinua es la que debe rechazarse: ni InfluxDB ni PostgreSQL publican su puerto y el Security Group no tiene reglas para 8086 ni 5432.

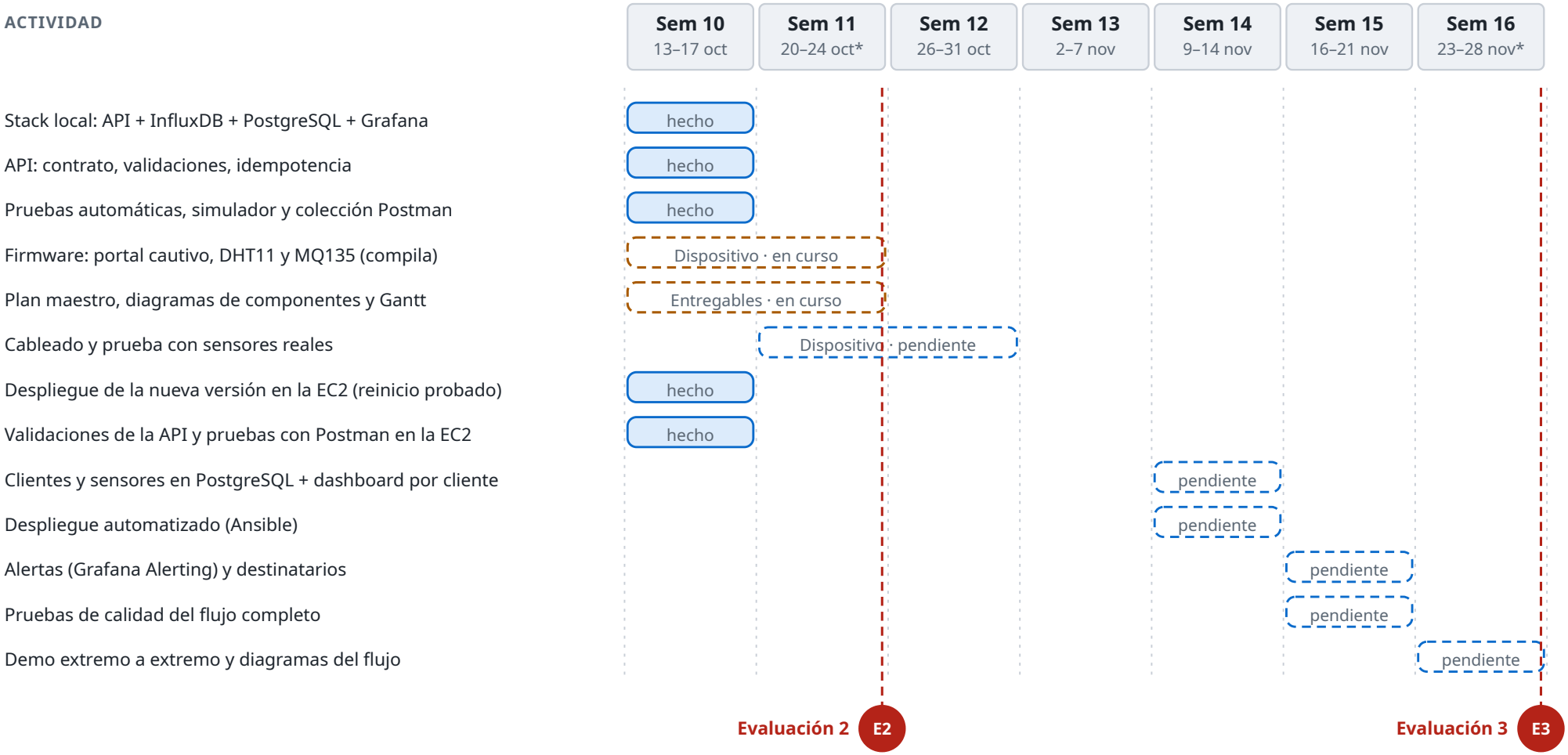
Anexo A.5 · Niveles de prueba



Orden de ejecución: 1 → 7. A = automatizado (pytest) · S = semi-automático · M = manual

Cada barra cubre los componentes que ese nivel integra. Los niveles se ejecutan de arriba abajo: primero el entorno, luego cada unión por separado y solo después el camino completo. Las barras punteadas (N5, N6) no añaden una unión nueva: someten al conjunto a fallos, carga y ataques.

Anexo A.6 · Gantt del proyecto



* Fechas de las semanas 11 y 16 estimadas a partir del calendario de las demás semanas.

Verde con borde azul = hecho; ámbar punteado = en curso; azul punteado = pendiente. Las líneas rojas marcan las evaluaciones 2 (semana 11) y 3 (semana 16). Las actividades siguen los hitos sugeridos en la clase «Del sensor a Grafana».